

Tema 6.

Análisis automático de sesgo léxico y discursivo

Sesgo lingüístico, léxicos transparentes, KWIC, coocurrencias y lectura atenta del discurso

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Dr.^a Blanca Hernández Pardo

Índice de contenidos

Contexto profesional.....	1
Encargo: una auditoría lingüística transparente.....	1
El caso de trabajo.....	1
Qué vas a aprender en esta unidad.....	2
Mapa conceptual de la unidad.....	2
1. Marco teórico.....	3
1.1. Por qué el sesgo lingüístico importa en PLN.....	3
1.2. Sesgo social, sesgo lingüístico, sesgo de datos y sesgo de sistema.....	4
1.3. Del análisis crítico del discurso al análisis asistido por corpus.....	4
1.4. Qué puede medir un enfoque léxico y qué no puede medir.....	5
1.5. Género, representación y sesgo en tecnologías lingüísticas.....	5
1.6. Lenguaje valorativo, polaridad y carga evaluativa.....	6
1.7. Preprocesamiento: limpiar también es interpretar.....	6
1.8. Tokenización, lematización y stopwords.....	7
1.9. Expresiones regulares como herramienta lingüística.....	7
1.10. KWIC y coocurrencias: del conteo al contexto.....	7
1.11. Datos reales, redes sociales y responsabilidad.....	8
1.12. Evaluar sesgo no es automatizar juicio moral.....	8
2. Análisis automático de sesgo léxico y discursivo con Python.....	8
2.1. Preparar el entorno de trabajo.....	8
2.2. Organizar el corpus en un DataFrame.....	9
2.3. Normalizar texto.....	10
2.4. Obtener stopwords en español.....	10
2.5. Detectar automáticamente la columna de texto.....	11
2.6. Tokenizar y lematizar.....	11
2.7. Construir un léxico de género marcado.....	12
2.8. Construir un léxico valorativo.....	12
2.9. Detectar términos relevantes en cada documento.....	13
2.10. Construir la tabla de resultados por texto.....	13
2.11. Calcular frecuencias globales.....	14
2.12. Ordenar textos por densidad de marcas.....	14
2.13. Construir concordancias KWIC.....	15
2.14. Calcular coocurrencias.....	15
2.15. Descargar y organizar un subcorpus real.....	15
2.16. Leer líneas y crear una muestra manejable.....	16
2.17. Crear un subcorpus temático con regex.....	16
2.18. Reutilizar el pipeline en datos reales.....	17
2.19. Agrupar resultados por etiqueta.....	17
2.20. Mejorar coocurrencias en redes sociales.....	18
2.21. Cómo escribir una interpretación responsable.....	18

Errores frecuentes de aprendizaje.....	19
Síntesis final.....	19
Cierre narrativo.....	19
Referencias.....	20

Título narrativo

Auditar sesgos en textos: mirar el lenguaje con lupa computacional y criterio lingüístico.

Contexto profesional

Encargo: una auditoría lingüística transparente

Imagina que te incorporas como lingüista computacional júnior a **Veridia Text Lab**, una consultora especializada en auditoría lingüística, análisis crítico del discurso y prototipos explicables de PLN para instituciones, medios de comunicación y equipos de comunicación pública. La empresa no vende sistemas que «descubren la verdad» de un texto: su valor profesional consiste en construir herramientas transparentes que ayudan a observar patrones lingüísticos que después interpretan especialistas humanos.

Veridia recibe un encargo de un consorcio de medios digitales y organizaciones sociales. El cliente quiere revisar titulares, mensajes breves y comentarios procedentes de entornos digitales para detectar indicios de:

- lenguaje excluyente;
- sesgos de género;
- exceso de léxico valorativo;
- asociaciones discursivas repetidas alrededor de determinados grupos sociales;
- posibles diferencias entre lo que una etiqueta de sentimiento dice y lo que el texto muestra lingüísticamente.

El cliente no pide una sentencia automática, sino un primer prototipo de exploración que permita localizar zonas de atención. La diferencia pesa mucho: una herramienta de este tipo no debe declarar que una persona «tuvo intención discriminatoria», ni que un texto «es sesgado» de manera absoluta; debe mostrar indicios observables —palabras, frecuencias, concordancias, coocurrencias y medidas sencillas— que permitan formular preguntas mejores.

El caso de trabajo

El equipo de Veridia propone una simulación profesional: construir un sistema básico de análisis automático de sesgo léxico y discursivo sobre textos breves en español. El sistema debe ser deliberadamente transparente y no utilizará aprendizaje automático complejo ni modelos generativos, porque en esta fase interesa comprender cómo se operacionaliza una pregunta lingüística, más que maximizar el rendimiento.

La simulación parte de una idea de fondo:

Un sesgo discursivo va más allá de una palabra aislada, pero muchas formas de sesgo dejan huellas en patrones léxicos, asociaciones recurrentes y formas de valoración.

Por eso el prototipo trabajará con cuatro operaciones principales:

1. Preparar los textos para que puedan procesarse.
2. Detectar términos de género y léxico valorativo mediante listas definidas de antemano.
3. Resumir patrones con tablas, frecuencias y puntuaciones simples.

4. Explorar contexto mediante KWIC y coocurrencias.

Más que actuar como una máquina de clasificación, tu papel consiste en aprender a construir una mirada computacional responsable: saber qué mide el código, qué no mide y por qué la interpretación lingüística sigue siendo imprescindible.

Qué vas a aprender en esta unidad

Al finalizar esta unidad, deberías ser capaz de:

- explicar qué significa estudiar sesgo lingüístico desde una perspectiva computacional;
- distinguir entre sesgo social, sesgo lingüístico, sesgo de datos y sesgo de sistema;
- comprender por qué un enfoque basado en reglas definidas de antemano puede ser útil en una auditoría inicial;
- preparar textos breves para análisis automático mediante normalización, tokenización, lematización y eliminación de *stopwords*;
- construir listas léxicas de términos de género y vocabulario valorativo;
- entender cómo funcionan las funciones de detección basadas en pertenencia a listas o diccionarios;
- interpretar qué representa una tabla de resultados por documento;
- calcular frecuencias globales con `Counter`;
- comprender la lógica de una concordancia KWIC;
- explicar qué es una coocurrencia y por qué depende de una ventana de contexto;
- filtrar un subcorpus temático con expresiones regulares;
- trabajar con un corpus real sin olvidar sus límites metodológicos y éticos;
- justificar por qué los resultados automáticos deben leerse como indicios, y no como veredictos.

Mapa conceptual de la unidad

Tabla 1. Mapa conceptual de la unidad: bloques, preguntas lingüísticas y operaciones

Bloque	Pregunta lingüística	Operación en Python	Resultado esperado
Corpus	¿Qué textos vamos a analizar?	Crear un DataFrame y extraer una columna textual	Una colección organizada de documentos
Normalización	¿Qué rasgos conservamos y cuáles eliminamos?	Minúsculas, limpieza con <code>re.sub</code> , espacios	Textos más comparables
Tokenización	¿Cuáles son las unidades mínimas de análisis?	Separar palabras con regex o <code>spaCy</code>	Listas de <i>tokens</i>
Lematización	¿Conviene agrupar formas flexionadas?	Usar <code>lemma</code> si hay modelo lingüístico	Formas más generales
Léxicos	¿Qué palabras consideramos relevantes?	Crear listas y diccionarios	Repertorios definidos
Detección	¿Aparecen esos términos en cada texto?	Comprensiones de listas y búsquedas en diccionarios	Marcas por documento
Agregación	¿Qué patrón se observa en el corpus?	<code>Counter</code> , DataFrame, ordenación	Frecuencias y resúmenes
KWIC	¿En qué contexto aparece una palabra?	Ventana izquierda y derecha	Concordancias
Coocurrencias	¿Qué palabras se asocian cerca de un término?	Conteo en ventanas de contexto	Listado de asociaciones
Interpretación	¿Qué podemos afirmar con rigor?	Comparar resultados y límites	Informe lingüístico prudente

Figura 1. Pipeline de análisis de sesgo léxico y discursivo



Nota. Elaboración propia a partir del mapa conceptual de la unidad.

1. Marco teórico

1.1. Por qué el sesgo lingüístico importa en PLN

El procesamiento del lenguaje natural trabaja con datos lingüísticos producidos por personas, instituciones, comunidades y plataformas, y esos datos no son neutrales: contienen hábitos expresivos, desigualdades sociales, convenciones culturales, estereotipos, silencios y formas de representación. Por eso, cuando un sistema computacional aprende o cuenta patrones en textos, opera sobre un lenguaje inscrito en un contexto social, nunca sobre una materia pura.

La investigación en PLN ha mostrado que los sistemas automáticos pueden reproducir y amplificar asociaciones presentes en los datos. Bolukbasi *et al.* (2016) pusieron en evidencia asociaciones estereotipadas en los *embeddings* distribucionales, y Caliskan *et al.* (2017) demostraron que los modelos entrenados sobre grandes corpus pueden recuperar sesgos humanos medibles. Estos trabajos no significan que todo análisis computacional sea dañino, pero obligan a preguntarse de dónde salen los datos, qué patrones contienen y qué consecuencias tendría automatizar esos patrones.

Blodgett *et al.* (2020) recuerdan que hablar de «sesgo» en PLN es, además de una cuestión técnica, una cuestión conceptual y normativa: hay que explicar qué comportamiento se considera problemático, para quién, en qué contexto y por qué. Esta idea pesa mucho para estudiantes de lingüística. El código puede contar palabras, comparar listas y construir tablas, pero la definición de lo que cuenta como exclusión, estereotipo o valoración injusta exige conocimiento lingüístico, sociocultural y ético.

En esta unidad, el análisis de sesgo se entiende como una práctica de observación. El sistema no detecta intenciones psicológicas ni decide si una comunidad «aparece bien» o «aparece mal» en un corpus; hace algo mucho más concreto: localiza formas léxicas, calcula frecuencias, observa contextos y organiza indicios. Esa modestia metodológica es una fortaleza, porque permite que el alumnado vea cada paso y pueda discutirlo.

1.2. Sesgo social, sesgo lingüístico, sesgo de datos y sesgo de sistema

Conviene distinguir varios niveles que a menudo se mezclan:

Tabla 2. Niveles de sesgo

Nivel	Definición	Ejemplo de pregunta
Sesgo social	Desigualdad, estereotipo o jerarquía existente en una comunidad	¿Qué grupos aparecen asociados a autoridad, cuidado, amenaza o incompetencia?
Sesgo lingüístico	Forma concreta en que esa desigualdad se manifiesta en palabras, estructuras o marcos discursivos	¿Qué adjetivos se aplican a hombres y mujeres en posiciones de poder?
Sesgo de datos	Desequilibrio o patrón problemático dentro del corpus utilizado	¿El corpus representa de forma desigual a ciertos grupos?
Sesgo de anotación	Efecto de las decisiones humanas al etiquetar datos	¿Dos anotadores interpretarían igual un comentario irónico?
Sesgo de sistema	Resultado problemático producido o amplificado por un modelo o herramienta	¿Un clasificador marca como ofensivo un dialecto o variedad de forma desproporcionada?

Sap *et al.* (2019) mostraron cómo la insensibilidad a la variación dialectal podía generar sesgos en la detección automática de discurso ofensivo. El problema iba más allá de la aparición de palabras «malas»: ciertos rasgos lingüísticos de una comunidad quedaban asociados a la toxicidad por la forma en que se habían anotado y modelado los datos. La enseñanza para esta unidad es clara: un patrón estadístico no equivale automáticamente a una interpretación social correcta.

También hay que distinguir entre sesgo representacional y sesgo asignativo. Un sesgo representacional afecta a cómo se describe, visibiliza o asocia a un grupo. Un sesgo asignativo afecta a decisiones o recursos: quién recibe una recomendación, una oportunidad o una clasificación favorable. En esta unidad trabajamos sobre todo con sesgo representacional: cómo aparecen ciertas palabras y qué vocabulario se acumula alrededor de determinados términos.

1.3. Del análisis crítico del discurso al análisis asistido por corpus

La lingüística de corpus y el análisis crítico del discurso ofrecen una base muy útil para esta unidad. Baker (2006) explica que las técnicas de corpus permiten observar regularidades en grandes colecciones de textos: frecuencias, concordancias, colocaciones, palabras clave y patrones de distribución. Baker *et al.* (2008) defienden la utilidad de combinar métodos de corpus con análisis crítico del discurso, porque los conteos ayudan a localizar patrones, pero la interpretación crítica explica su relevancia social.

Esta combinación es exactamente la que necesita una lingüista computacional. Que una palabra aparezca muchas veces no dice todavía qué significa discursivamente: puede ser una marca de tema, un rasgo de género textual, una repetición banal o un indicio de representación problemática. La frecuencia orienta la mirada, pero no sustituye el análisis.

En el enfoque asistido por corpus, tres herramientas tienen un peso especial:

Tabla 3. Herramientas del análisis asistido por corpus

Herramienta	Qué muestra	Para qué sirve en análisis de sesgo
Frecuencia	Cuántas veces aparece una forma	Detectar presencia, ausencia o sobrerrepresentación

KWIC	Una palabra con su contexto inmediato	Observar cómo se usa una forma en ejemplos concretos
Coocurrencia	Palabras que aparecen cerca de otra	Explorar asociaciones recurrentes

McEnery y Hardie (2012) insisten en que el corpus debe analizarse con atención a su diseño: tamaño, origen, muestreo, género textual, representatividad y permisos. Esta cautela será importante cuando pasemos de un minicorpus didáctico a un subcorpus real de redes sociales.

1.4. Qué puede medir un enfoque léxico y qué no puede medir

Un enfoque léxico parte de una idea sencilla: si tenemos una lista de palabras relevantes, podemos buscar esas palabras en los textos y contar cuántas aparecen. En análisis de sentimiento, esta lógica se usa desde hace décadas para detectar vocabulario positivo o negativo. Hu y Liu (2004) trabajaron con opiniones de clientes y polaridad, y Liu (2012) consolidó el análisis de sentimiento y minería de opiniones como un área central del PLN aplicado. Mohammad y Turney (2013) ampliaron esta tradición con recursos léxicos de emoción y polaridad.

La ventaja de un enfoque léxico es su transparencia: si el sistema ha marcado una palabra, se puede saber exactamente por qué. Esto es valioso en la docencia y en la auditoría, porque permite revisar el léxico, ampliarlo, eliminar términos problemáticos y explicar el resultado a personas no técnicas.

La limitación es igual de importante. Una lista no entiende de ironía, negación compleja, presuposiciones, metáforas, cambios de registro ni contexto pragmático. Si un texto dice «qué comentario tan brillante» de forma irónica, una lista puede interpretar *brillante* como positivo aunque el acto comunicativo sea negativo, y si un texto evalúa a una persona sin utilizar palabras del diccionario, el sistema no verá esa valoración. Por eso las medidas léxicas deben leerse como indicadores de superficie, y no como un diagnóstico completo.

En el análisis de sesgo, esta distinción es esencial. Una palabra como *madre*, *jefe* o *candidata* no es sesgada por sí misma: puede ser una mención neutra, una marca gramatical necesaria o parte de una construcción estereotipada. El análisis computacional ayuda a localizar dónde mirar; la lectura lingüística decide cómo interpretar.

1.5. Género, representación y sesgo en tecnologías lingüísticas

El sesgo de género en PLN se ha estudiado en distintos niveles: representación en corpus, asociaciones en *embeddings*, predicciones de modelos, generación automática y evaluación de plantillas. Bolukbasi *et al.* (2016) mostraron asociaciones de género en vectores de palabras; Caliskan *et al.* (2017) relacionaron esos patrones con sesgos humanos más amplios, y Sheng *et al.* (2019) analizaron sesgos en la generación de lenguaje a partir de menciones demográficas.

En español, el problema presenta rasgos específicos. La lengua tiene género gramatical en sustantivos, adjetivos, determinantes y participios, lo que no convierte toda marca de género en algo socialmente problemático, pero sí obliga a extremar el cuidado en el análisis automático. Palabras como *estudiante* o *deportista* pueden funcionar como comunes en cuanto al género, mientras que formas como *candidato/candidata* o *jefe/jefa* expresan una alternancia morfológica marcada. Garrido-Muñoz *et al.* (2024) estudian el sesgo de género en modelos de lengua para el español y muestran que su evaluación en esta lengua necesita métodos y recursos adaptados, y no simples traducciones de recursos en inglés.

Para una lingüista, esto tiene una consecuencia metodológica: no basta con preguntar «¿aparece una palabra masculina o femenina?», hay que mirar funciones discursivas, papeles semánticos, adjetivación, verbos asociados y contexto. Un análisis de listas puede detectar candidatos para revisar, pero no puede resolver por sí solo si existe invisibilización, estereotipo o asimetría de trato.

1.6. Lenguaje valorativo, polaridad y carga evaluativa

El léxico valorativo es el conjunto de palabras que expresan evaluación, afecto, juicio o intensidad: adjetivos como *excelente*, *terrible*, *agresivo* o *responsable*; sustantivos como *desastre* o *éxito*; verbos como *amenazar*, *apoyar* o *minimizar*, y expresiones fraseológicas más complejas. En esta unidad se trabaja con una versión simplificada: un diccionario que asigna polaridad positiva o negativa a determinadas palabras.

La polaridad es una etiqueta técnica (positivo, negativo o neutro), pero la valoración discursiva es más rica: una palabra positiva puede funcionar con ironía; una palabra aparentemente neutra puede adquirir carga negativa por el contexto, y una ausencia puede ser discursivamente significativa. Por eso la polaridad léxica sirve como primer filtro, nunca como interpretación definitiva.

En entornos de redes sociales esta dificultad aumenta. Los textos son breves, mezclan registros, omiten contexto, usan abreviaturas, emojis, *hashtags* y referencias culturales. Pérez *et al.* (2022), al presentar RoBERTuito para texto de redes sociales en español, muestran que el dominio importa: los textos de redes tienen características distintas de textos periodísticos, académicos o administrativos. Aunque en esta unidad no se entrena un modelo, sí se hereda esa lección: el tipo de corpus condiciona qué significa limpiar, tokenizar e interpretar.

1.7. Preprocesamiento: limpiar también es interpretar

En PLN se llama preprocesamiento al conjunto de operaciones que transforman un texto bruto en una representación más manejable para el análisis. Puede incluir:

- pasar a minúsculas;
- eliminar URLs;
- eliminar menciones de usuario;
- limpiar *hashtags*;
- quitar números;
- retirar puntuación;
- normalizar espacios;
- tokenizar;
- lematizar;
- eliminar *stopwords*.

Estas operaciones parecen técnicas, pero tienen implicaciones lingüísticas. Si se elimina el símbolo # de un *hashtag* y se conserva la palabra, se decide que el contenido léxico importa más que la marca de plataforma. Si se eliminan emojis, se puede perder información afectiva. Si se eliminan menciones de usuario, se protege privacidad y se reduce ruido, pero también se pierde información interactiva. Si se eliminan *stopwords*, se puede mejorar la lectura de coocurrencias, pero también desaparecerán conectores, negaciones o marcas pragmáticas relevantes.

Bender y Friedman (2018) proponen documentar los datos lingüísticos para evitar generalizaciones indebidas, y Gebru *et al.* (2021) defienden las fichas técnicas de conjuntos de datos (*datasheets for datasets*) como práctica de transparencia. En una unidad como esta, esa idea puede trasladarse al

aula: cada decisión de limpieza debería poder justificarse con una frase, porque limpiar responde a la decisión de qué parte del texto se quiere analizar, y no a un «hay que limpiar» genérico.

1.8. Tokenización, lematización y stopwords

La tokenización separa el texto en unidades, que en un análisis sencillo suelen ser palabras. Pero incluso esta operación exige decisiones: ¿qué hacemos con los signos de puntuación?, ¿con los números?, ¿con los *hashtags*?, ¿con los emojis?, ¿con las palabras con tilde?, ¿con las formas compuestas?

La lematización reduce una forma flexionada a su lema. Por ejemplo, *candidatas* y *candidata* pueden agruparse bajo una forma común si el analizador lingüístico funciona bien, algo útil en español para no tratar cada flexión como una palabra completamente distinta. Sin embargo, la lematización también puede ocultar información: si el objetivo es estudiar las marcas morfológicas de género, perder la forma superficial puede ser un problema. En esta unidad se acepta ese coste porque el objetivo es didáctico: comparar el funcionamiento de un *pipeline* con spaCy y sin él.

Las *stopwords* son palabras frecuentes que muchas veces aportan poca información temática: determinantes, preposiciones, conjunciones y algunas formas auxiliares. Eliminarlas puede hacer más legible una tabla de coocurrencias, pero no siempre conviene, porque en el análisis discursivo palabras pequeñas como *no*, *pero* o *aunque* pueden ser centrales. La regla profesional es sencilla: una *stopword* solo es irrelevante para una tarea concreta y bajo una justificación concreta, nunca por naturaleza.

1.9. Expresiones regulares como herramienta lingüística

Las expresiones regulares permiten buscar patrones formales en cadenas de texto, y en Python se usan mediante el módulo `re` (Python Software Foundation, 2026). Para lingüistas, una regex puede entenderse como una microgramática de búsqueda: no describe toda la lengua, pero sí una familia de formas que queremos localizar. Por ejemplo, un patrón puede buscar URL, menciones, números o una lista de términos de género. El símbolo `\b` marca límites de palabra y ayuda a evitar que una búsqueda encuentre una secuencia dentro de otra palabra; el operador `|` funciona como «o», y el prefijo `r` crea una cadena cruda, útil porque así las barras invertidas de una regex no se interpretan como escapes normales de Python.

Las regex son muy potentes, pero también frágiles, porque capturan formas y no significados. Si buscamos *mujer* y *mujeres*, encontraremos esas formas, pero si el texto habla de género mediante una perífrasis, no aparecerá; si contiene errores ortográficos, abreviaturas o variantes dialectales, el patrón puede fallar. Por eso las regex son excelentes para prototipos transparentes, siempre que se revisen lingüísticamente.

1.10. KWIC y coocurrencias: del conteo al contexto

Una concordancia KWIC (*Key Word In Context*) muestra una palabra clave con algunas palabras a su izquierda y a su derecha. Es una herramienta clásica de corpus porque permite pasar de la frecuencia al uso contextual: si una forma aparece veinte veces, el KWIC permite leer rápidamente en qué entornos lo hace.

Las coocurrencias cuantifican las palabras que aparecen cerca de una palabra objetivo. Si alrededor de una mención de género aparecen de forma recurrente adjetivos de competencia, emoción, amenaza o autoridad, esa asociación puede orientar una lectura discursiva. Pero la coocurrencia no prueba

causalidad ni intención: solo dice que ciertas palabras aparecen cerca bajo una definición concreta de ventana.

La ventana es una decisión metodológica. Una ventana de dos *tokens* captura contexto muy inmediato; una de ocho *tokens* captura más material, pero también introduce más ruido. En textos breves de redes sociales, una ventana grande puede abarcar casi todo el mensaje. En textos largos, una ventana pequeña puede perder unidades sintácticas completas. Por eso, la ventana debe elegirse pensando en el género textual.

1.11. Datos reales, redes sociales y responsabilidad

Trabajar con datos reales aumenta el valor formativo del análisis, pero también sus riesgos. Un minicorpus didáctico está controlado: permite que el alumnado comprenda el *pipeline* sin depender de descargas, licencias o fenómenos demasiado dispersos. Un corpus real introduce variación, ruido, errores, ambigüedad, desequilibrio y problemas éticos.

Los datos de redes sociales son muy delicados. Aunque un texto sea técnicamente accesible, puede contener información personal, lenguaje sensible o contextos que se pierden al aislar el mensaje, y la propia plataforma condiciona el lenguaje: abreviaturas, retuits, menciones, *hashtags*, emojis y fórmulas de interacción forman parte del significado. En una auditoría profesional no basta con descargar textos: hay que documentar procedencia, permisos, sesgos de muestreo, periodo temporal, idioma, anonimización y finalidad del análisis.

En esta unidad, el corpus real se usa como subcorpus manejable y filtrado, una elección que enseña una práctica profesional: en lugar de analizarlo todo indiscriminadamente, se formula una pregunta, se selecciona una muestra y se explican los límites.

1.12. Evaluar sesgo no es automatizar juicio moral

El punto más importante de este marco teórico es ético y metodológico: detectar patrones no equivale a emitir un juicio moral automático. Gallegos *et al.* (2024) sintetizan la investigación reciente sobre sesgo y equidad en modelos de lenguaje y muestran lo complejo que resulta medir, evaluar y mitigar daños; incluso en sistemas mucho más avanzados que el de esta unidad, la medición del sesgo depende de definiciones, grupos, contextos, métricas y objetivos. Por eso un prototipo transparente de reglas tiene valor pedagógico: obliga a ver qué hace el sistema. Si detecta léxico de género, es porque ese léxico está en una lista; si calcula una puntuación de valoración, es porque se ha asignado +1 o -1 a determinadas palabras, y si muestra coocurrencias, es porque se ha definido una ventana. Nada queda escondido en una caja negra.

La responsabilidad lingüística consiste en escribir informes con prudencia. En lugar de decir «el texto es sesgado», conviene decir «el texto contiene léxico valorativo asociado a menciones de género» o «en este subcorpus se observan coocurrencias que conviene revisar cualitativamente». Esta diferencia de formulación es una diferencia profesional.

2. Análisis automático de sesgo léxico y discursivo con Python

2.1. Preparar el entorno de trabajo

La unidad utiliza un conjunto pequeño de bibliotecas, porque busca un *pipeline* que pueda ejecutarse en un aula y que se comprenda línea a línea, más que un sistema pesado.

```
import pandas as pd
import re
from collections import Counter, defaultdict
```

`pandas` permite organizar los textos y los resultados en tablas. Para una lingüista, un `DataFrame` puede entenderse como una hoja de cálculo programable, en la que cada fila representa un texto y cada columna, una propiedad de ese texto.

`re` permite trabajar con expresiones regulares; en esta unidad se usa para limpiar texto, tokenizar de forma básica y filtrar subcorpus por términos de interés.

`Counter` sirve para contar los elementos de una lista: si tenemos una lista de *tokens*, devuelve cuántas veces aparece cada uno. `defaultdict` se importa como herramienta de apoyo para estructuras de conteo o agrupación, aunque el núcleo de la unidad puede resolverse principalmente con `Counter`.

El entorno también intenta cargar `spaCy` de forma opcional:

```
try:
    import spacy
    try:
        nlp = spacy.load("es_core_news_sm")
    except Exception:
        nlp = None
except Exception:
    spacy = None
    nlp = None
```

Este bloque enseña una práctica profesional muy útil: escribir código que resista los fallos. Si `spaCy` y el modelo de español están disponibles, el análisis puede usar la lematización y las *stopwords* del modelo (Explosion AI, 2026); si no lo están, el programa no se rompe y activa una alternativa basada en `regex` y listas de *stopwords*. En el contexto docente, esto evita que una dependencia técnica bloquee la comprensión conceptual.

2.2. Organizar el corpus en un `DataFrame`

El primer paso de cualquier análisis textual es decidir qué se considera documento. En esta unidad, cada texto breve se trata como un documento. Los documentos se guardan en una lista y después se convierten en una tabla:

```
df = pd.DataFrame({"text": corpus})
```

Esta línea crea una tabla con una columna llamada `text`, en la que cada fila contiene un texto. A partir de ese momento, el análisis puede añadir nuevas columnas: *tokens*, términos de género encontrados, términos valorativos, puntuaciones o etiquetas.

El manual no necesita entrar en los casos concretos del corpus, que se discutirán en el aula. Lo importante antes de programar es entender la estructura: el texto original en una columna, las representaciones procesadas en columnas nuevas y los resultados en una tabla final.

Una operación habitual es extraer la columna textual como lista:

```
textos = df["text"].tolist()
```

`df["text"]` selecciona una columna y `.tolist()` la convierte en una lista de Python, una transformación útil porque muchas funciones de PLN reciben una lista de textos o recorren los textos uno a uno.

2.3. Normalizar texto

La función de normalización transforma cada texto antes de tokenizarlo. Su objetivo es reducir variación superficial que puede impedir comparaciones.

```
def normalizar_texto(texto):
    texto = texto.lower()
    texto = re.sub(r"http\S+|www\S+", "", texto)
    texto = re.sub(r"@w+", "", texto)
    texto = re.sub(r"#", "", texto)
    texto = re.sub(r"d+", "", texto)
    texto = re.sub(r"[^\w\s]", "", texto)
    texto = re.sub(r"s+", " ", texto).strip()
    return texto
```

La primera línea, `texto.lower()`, convierte todo a minúsculas, de modo que *Mujer*, *mujer* y *MUJER* no se tratan como tres formas distintas. Esta decisión mejora el conteo, pero elimina la información sobre mayúsculas enfáticas o nombres propios.

Las llamadas a `re.sub` sustituyen patrones por una cadena vacía. El patrón `r"http\S+|www\S+"` localiza URL; `r"@w+"`, menciones de usuario; `r"#"` elimina el símbolo de *hashtag*, aunque no necesariamente la palabra que lo acompaña; `r"d+"` elimina números, y `r"[^\w\s]"`, la puntuación y los símbolos que no sean caracteres de palabra ni espacios. Por último, `r"s+"` agrupa los espacios repetidos y `.strip()` elimina los del principio y del final.

La normalización se aplica con una comprensión de listas:

```
textos_norm = [normalizar_texto(t) for t in textos]
```

Esta línea significa «para cada texto `t` dentro de `textos`, aplica la función `normalizar_texto` y guarda el resultado». La comprensión de listas es compacta, pero conceptualmente equivale a un bucle.

2.4. Obtener stopwords en español

La unidad define una función `get_stopwords_es()` que intenta conseguir *stopwords* en español siguiendo un orden de prioridad:

1. Primero intenta usar spaCy.
2. Si spaCy no está disponible, intenta usar NLTK.
3. Si tampoco se puede usar NLTK, recurre a una lista mínima interna.

Este diseño enseña una idea importante: un buen código docente debe funcionar también fuera del ordenador ideal, previendo fallos razonables y ofreciendo alternativas.

```
def get_stopwords_es():
    try:
        from spacy.lang.es.stop_words import STOP_WORDS as SPACY_STOP
        return set(SPACY_STOP)
    except Exception:
        pass

    try:
        import nltk
        from nltk.corpus import stopwords
        try:
            return set(stopwords.words("spanish"))
        except LookupError:
```

```
try:
    nltk.download("stopwords", quiet=True)
    return set(stopwords.words("spanish"))
except Exception:
    pass
except Exception:
    pass

return {"de", "la", "que", "el", "en", "y", "a", "los"}
```

La función devuelve un `set`, y no una lista, porque en Python buscar si una palabra está dentro de un conjunto suele ser más eficiente que buscarla dentro de una lista. Para esta unidad el tamaño no es enorme, pero la elección enseña una buena práctica.

2.5. Detectar automáticamente la columna de texto

Los conjuntos de datos no siempre llaman igual a la columna textual: un archivo puede tener `text`; otro, `texto`; otro, `tweet`, y otro, `content`. Para evitar que el código dependa de un único nombre, se define una pequeña heurística:

```
def detectar_columna_texto(df):
    candidatos = ["texto", "text", "tweet", "tweets", "content", "sentence", "review"]
    for c in candidatos:
        if c in df.columns:
            return c
    obj_cols = [c for c in df.columns if df[c].dtype == "object"]
    if obj_cols:
        return obj_cols[0]
    raise KeyError("No he encontrado una columna de texto en el DataFrame.")
```

La función revisa una lista de nombres probables y, si encuentra uno, lo devuelve. Si no, busca la primera columna de tipo `object`, que en pandas suele corresponder a texto, y si tampoco existe, lanza un error claro.

Esta función no es lingüísticamente profunda, pero sí profesional: hace que el *pipeline* sea reutilizable con distintos corpus y enseña que los datos reales rara vez llegan con nombres perfectos y homogéneos.

2.6. Tokenizar y lematizar

La unidad define una tokenización simple con regex:

```
def tokenizar(texto: str):
    return re.findall(r"[a-záéíóúñ]+", str(texto).lower())
```

El patrón busca secuencias de letras minúsculas, incluidas las vocales acentuadas, la diéresis y la *ñ*, una decisión adaptada al español que deja fuera emojis, signos y números. Para el objetivo de la unidad, esto facilita la lectura de las listas léxicas; para otros objetivos, podría resultar insuficiente.

Después aparece una función más completa:

```
def tokenizar_y_lematizar(texto: str):
    if nlp is None:
        toks = tokenizar(texto)
        return [t for t in toks if t not in STOPWORDS_ES and len(t) > 2]
```

```
doc = nlp(str(texto).lower())
return [
    t.lemma_
    for t in doc
    if (not t.is_stop) and (not t.is_space) and (t.is_alpha) and len(t.lemma_) > 2
]
```

La función tiene dos caminos: si no hay modelo de spaCy, tokeniza con regex y elimina *stopwords* y *tokens* muy cortos; si lo hay, procesa el texto y devuelve lemas. En ambos casos se busca una representación más limpia que el texto original.

La línea final de aplicación es:

```
tokens = [tokenizar_y_lematizar(t) for t in df[TEXT_COL].tolist()]
df["tokens"] = tokens
```

El resultado es una columna nueva llamada `tokens`, en la que cada fila contiene una lista de palabras procesadas, base de casi todo el análisis posterior.

2.7. Construir un léxico de género marcado

El léxico de género se representa mediante un diccionario de pares:

```
dic_genero = {
    "hombre": "mujer",
    "mujer": "hombre",
    "jefe": "jefa",
    "jefa": "jefe",
    "candidato": "candidata",
    "candidata": "candidato",
    "estudiante": "estudiante",
    "deportista": "deportista"
}

lexico_genero = sorted(set(dic_genero.keys()) | set(dic_genero.values()))
```

El diccionario permite guardar relaciones entre formas: algunas son pares masculino/femenino, y otras, como *estudiante* o *deportista*, se mantienen como formas comunes. Después se combinan claves y valores con una unión de conjuntos, y el resultado, `lexico_genero`, es una lista ordenada de términos que se buscarán en los textos.

Desde el punto de vista lingüístico, este léxico no pretende resolver el género en español: es un recurso didáctico y ampliable, que sirve para aprender a formular una hipótesis con precisión («estas son las formas que voy a considerar relevantes en esta fase del análisis»).

2.8. Construir un léxico valorativo

El léxico valorativo se organiza en dos listas y un diccionario final:

```
lexico_valorativo_positivo = ["excelente", "bueno", "brillante", "responsable"]
lexico_valorativo_negativo = ["terrible", "agresivo", "incompetente", "desastre"]

dic_valorativo = {w.lower(): "positivo" for w in lexico_valorativo_positivo}
dic_valorativo.update({w.lower(): "negativo" for w in lexico_valorativo_negativo})
```

La primera comprensión de diccionario asigna la etiqueta `positivo` a cada palabra de la lista positiva, y después `.update()` añade las palabras negativas con la etiqueta `negativo`.

El resultado es una estructura de consulta: si una palabra aparece como clave, el sistema puede recuperar su polaridad. Por ejemplo, `dic_valorativo["excelente"]` devuelve `positivo`. Esta estructura es más útil que dos listas separadas cuando queremos calcular una puntuación.

La puntuación que se usará después es deliberadamente simple: +1 para los términos positivos y -1 para los negativos. Esta escala no mide la intensidad real; solo crea una señal inicial para ordenar y comparar textos.

2.9. Detectar términos relevantes en cada documento

Las funciones de detección son muy cortas:

```
def detectar_lexico_genero(tokens_doc, lexico_genero):
    return [t for t in tokens_doc if t in lexico_genero]

def detectar_lexico_valorativo(tokens_doc, dic_valorativo):
    return [t for t in tokens_doc if t in dic_valorativo]
```

Cada función recorre los *tokens* de un documento y conserva solo los que aparecen en el recurso correspondiente. La primera consulta una lista de términos de género. La segunda consulta las claves del diccionario valorativo.

Estas funciones no clasifican textos: no dicen «este texto es sesgado» ni «este texto es negativo», solo devuelven evidencias léxicas, esto es, qué términos de interés han aparecido. Esta diferencia debe quedar muy clara antes de interpretar resultados.

2.10. Construir la tabla de resultados por texto

El bloque central recorre todos los textos tokenizados y genera un diccionario de resultados para cada documento:

```
resultados = []
for i, texto_tokens in enumerate(tokens):
    genero = detectar_lexico_genero(texto_tokens, lexico_genero)
    valorativo = detectar_lexico_valorativo(texto_tokens, dic_valorativo)

    score_valorativo = sum(1 if dic_valorativo[t] == "positivo" else -1 for t in
                           valorativo)

    resultados.append({
        "texto_id": i + 1,
        "texto": textos[i],
        "tokens": texto_tokens,
        "lexico_genero": genero,
        "n_genero": len(genero),
        "lexico_valorativo": valorativo,
        "n_valorativo": len(valorativo),
        "score_valorativo": score_valorativo
    })

df_resultados = pd.DataFrame(resultados)
```

`enumerate(tokens)` permite recorrer la lista de documentos y conservar a la vez el índice `i`, que se usa para recuperar el texto original y crear un identificador.

La variable `genero` guarda las marcas de género detectadas, y `valorativo`, las palabras valorativas. `len(genero)` y `len(valorativo)` cuentan cuántas marcas hay, y `score_valorativo` suma +1 o -1 según la polaridad.

El resultado final, `df_resultados`, es una tabla analítica en la que cada fila representa un texto y cada columna, una observación. Esta estructura permite ordenar, filtrar, agrupar y revisar ejemplos concretos sin perder el vínculo con el texto original.

2.11. Calcular frecuencias globales

Para observar el patrón global del corpus, se juntan todos los *tokens*:

```
todos_tokens = [t for sublist in df_resultados["tokens"].tolist() for t in sublist]
frecuencias = Counter(todos_tokens)
```

La comprensión de listas tiene dos niveles: primero recorre cada lista de *tokens* y después cada *token* dentro de esa lista. El resultado es una lista plana con todos los *tokens* del corpus, y `Counter` cuenta cuántas veces aparece cada uno.

Después se construye una tabla solo con términos relevantes:

```
frecuencias_sesgo = pd.DataFrame([
    {"palabra": p,
     "frecuencia": frecuencias[p],
     "tipo": ("género" if p in lexico_genero else "valorativo"),
     "polaridad": dic_valorativo.get(p, "")}
    for p in sorted(set(lexico_genero) | set(dic_valorativo.keys()))
    if frecuencias[p] > 0
]).sort_values(["tipo", "frecuencia"], ascending=[True, False])
```

Esta construcción combina tres ideas: selección de términos de interés, consulta de frecuencia y creación de filas para un `DataFrame`. La columna `tipo` distingue si la palabra pertenece al léxico de género o al valorativo, y la columna `polaridad` solo tendrá contenido cuando el término esté en el diccionario valorativo.

La ordenación final permite ver primero los grupos y, dentro de ellos, las palabras más frecuentes. Una vez más, la frecuencia no interpreta por sí sola: organiza la observación.

2.12. Ordenar textos por densidad de marcas

Una operación frecuente en auditoría lingüística es seleccionar textos que merecen revisión cualitativa. Para eso se puede ordenar la tabla por número de marcas:

```
top_genero = df_resultados.sort_values(
    ["n_genero", "n_valorativo"],
    ascending=False
).head(8)
```

Esta línea no identifica los textos «peores» ni «más sesgados»; selecciona los textos con más marcas según los criterios definidos. La diferencia importa, porque el código no sabe qué es un problema social y se limita a aplicar una estrategia de priorización.

El método `sort_values` ordena por una o varias columnas; `ascending=False` indica orden descendente, y `.head(8)` devuelve las ocho primeras filas. En una auditoría real, este tipo de selección permite decidir qué ejemplos revisar primero.

2.13. Construir concordancias KWIC

La función KWIC crea una tabla con contexto izquierdo y derecho:

```
def kwic(tokens_corpus, term, window=4, max_lines=12):
    lines = []
    for doc_id, toks in enumerate(tokens_corpus, start=1):
        for i, t in enumerate(toks):
            if t == term:
                left = " ".join(toks[max(0, i-window):i])
                right = " ".join(toks[i+1:i+1+window])
                lines.append({"texto_id": doc_id, "izq": left, "term": term, "dcha":
right})
    return pd.DataFrame(lines).head(max_lines)
```

La función recorre documento por documento y *token* por *token*, y cuando encuentra el término buscado, toma una ventana de palabras a su izquierda y a su derecha. `max(0, i-window)` evita que el índice izquierdo sea negativo cuando el término aparece al principio del texto.

`" ".join(...)` reconstruye una secuencia de *tokens* como texto legible. La tabla resultante tiene cuatro columnas: identificador del texto, contexto izquierdo, término y contexto derecho.

El KWIC sirve para leer usos: es una herramienta puente, que parte de una búsqueda automática pero obliga a volver al contexto lingüístico.

2.14. Calcular coocurrencias

La coocurrencia cuenta qué palabras aparecen cerca de un término objetivo:

```
def coocurrencias(tokens_corpus, target, window=4):
    co = Counter()
    for toks in tokens_corpus:
        for i, t in enumerate(toks):
            if t == target:
                left = toks[max(0, i-window):i]
                right = toks[i+1:i+1+window]
                for w in left + right:
                    if w != target:
                        co[w] += 1
    return co
```

`co` empieza como un contador vacío. Cada vez que el sistema encuentra el término objetivo, recoge las palabras cercanas y suma una ocurrencia para cada una, y el resultado puede convertirse en una tabla con las palabras más frecuentes.

La coocurrencia no entiende de sintaxis: no sabe si una palabra modifica a otra, si pertenece a otra oración o si aparece por azar. Aun así, ayuda a detectar asociaciones recurrentes que después pueden revisarse con KWIC o con una lectura cualitativa.

2.15. Descargar y organizar un subcorpus real

La unidad incorpora una fase con datos reales descargables. El código usa `os` para crear una carpeta y `urlretrieve` para descargar los archivos si no existen:

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

```
import os
from urllib.request import urlretrieve

DATA_DIR = "data_subcorpus_real"
os.makedirs(DATA_DIR, exist_ok=True)

base = "https://raw.githubusercontent.com/garnachod/TwitterSentimentDataset/master/"
archivos = {
    "pos": "tweets_pos_clean.txt",
    "neg": "tweets_neg_clean.txt",
    "neu": "tweets_clean.txt"
}
```

La estructura `archivos` es un diccionario en el que cada clave representa una etiqueta y cada valor, un nombre de archivo. Después, el código recorre ese diccionario, construye rutas locales y descarga solo lo necesario.

Esta parte enseña dos ideas profesionales. Primero, no conviene depender de descargas repetidas: si un archivo ya existe, se reutiliza. Segundo, un corpus real suele venir distribuido en archivos, carpetas y etiquetas; antes de analizarlo hay que convertirlo en una estructura de trabajo.

2.16. Leer líneas y crear una muestra manejable

La función `leer_lineas` abre un archivo y devuelve una lista de líneas no vacías:

```
def leer_lineas(path, max_lines=None):
    lineas = []
    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        for i, line in enumerate(f):
            line = line.strip()
            if line:
                lineas.append(line)
            if max_lines is not None and len(lineas) >= max_lines:
                break
    return lineas
```

`with open(...)` as `f` abre el archivo de forma segura; `encoding="utf-8"` indica la codificación; `errors="ignore"` evita que un carácter problemático detenga toda la lectura, y `max_lines` permite limitar el número de líneas, algo muy útil en clase para que el análisis sea rápido.

Después se crea una tabla con textos y etiquetas:

```
df_real = pd.DataFrame({
    "texto": pos + neg + neu,
    "etiqueta": (["pos"] * len(pos)) + (["neg"] * len(neg)) + (["neu"] * len(neu))
}).reset_index(drop=True)
```

La columna `texto` contiene los mensajes, y la columna `etiqueta` conserva la clase original. Conviene no confundir esa etiqueta con el resultado del análisis léxico: la etiqueta viene del conjunto de datos, mientras que las medidas de la unidad se calculan después.

2.17. Crear un subcorpus temático con regex

Para estudiar un fenómeno poco frecuente, a veces se filtra el corpus. En esta unidad se buscan textos que contienen términos de género concretos:

```
pat = re.compile(
```

```
    r"\b(?:mujer|mujeres|hombre|hombres|jefa|jefas|jefe|jefes|chica|chicas|chico|chicos)
\b",
    re.IGNORECASE
)

df_sub = df_real[df_real["texto"].astype(str).str.contains(pat, regex=True)].copy()
```

`re.compile` prepara el patrón; `re.IGNORECASE` hace que la búsqueda no distinga mayúsculas y minúsculas; `(?:...)` agrupa alternativas sin crear un grupo de captura, y `\b` marca los límites de palabra.

El método `str.contains` aplica el patrón a cada texto y devuelve una serie booleana (verdadero si el texto contiene alguna forma, falso si no), que se usa para filtrar `df_real`. `.copy()` crea una copia independiente y evita advertencias de pandas cuando más adelante se modifiquen columnas.

El subcorpus temático no es más representativo que el corpus general, sino más pertinente para una pregunta concreta, y esa distinción debe aparecer en cualquier informe.

2.18. Reutilizar el *pipeline* en datos reales

Una buena señal de diseño es que las funciones creadas al principio puedan reutilizarse:

```
tokens_sub = [tokenizar_y_lematizar(t) for t in df_sub["texto"].tolist()]

resultados_sub = []
for i, texto_tokens in enumerate(tokens_sub):
    genero = detectar_lexico_genero(texto_tokens, lexico_genero)
    valorativo = detectar_lexico_valorativo(texto_tokens, dic_valorativo)
    score = sum(1 if dic_valorativo.get(t) == "positivo" else -1
                for t in valorativo if t in dic_valorativo)
```

El mismo patrón se repite: tokenizar, detectar, contar, puntuar y guardar resultados, con la diferencia de que ahora los textos proceden de un corpus real y tienen una etiqueta externa.

La tabla final del subcorpus incluye columnas como `etiqueta`, `texto`, `tokens`, `genero_encontrado`, `valorativo_encontrado`, `n_genero`, `n_valorativo` y `score_valorativo`, una estructura que permite comparar dos capas:

- la etiqueta original del conjunto de datos;
- las marcas léxicas detectadas por nuestro sistema.

Si ambas capas no coinciden, no significa necesariamente que una esté mal: significa que miden cosas distintas.

2.19. Agrupar resultados por etiqueta

Para resumir los resultados por clase se usa `groupby`:

```
resumen = df_resultados_sub.groupby("etiqueta") [
    ["n_genero", "n_valorativo", "score_valorativo"]
].mean().round(2)
```

`groupby("etiqueta")` divide la tabla según los valores de la columna `etiqueta`; después se seleccionan las columnas numéricas y se calcula la media, y `.round(2)` redondea a dos decimales.

Este resumen no debe interpretarse como una verdad psicológica sobre los tuits: solo muestra promedios dentro del subcorpus filtrado y bajo los léxicos definidos. En un informe habría que acompañarlo del tamaño de muestra, los criterios de filtrado y las advertencias sobre representatividad.

2.20. Mejorar coocurrencias en redes sociales

En el subcorpus real, las coocurrencias pueden llenarse de ruido, y por eso se define una función más cuidadosa:

```
def tok_tweet(s):
    return re.findall(r"[a-záéíóúñ]+", str(s).lower())

def coocurrencias_df(texts, target_terms, window_tokens=6, top_n=20,
                    stopwords=None, extra_stop=None, min_len=3):
    targets = set(t.lower() for t in target_terms)
    stop = set(stopwords or [])
    stop |= set(extra_stop or [])
    counts = Counter()
```

La función convierte los términos objetivo y las *stopwords* en conjuntos; después recorre los textos, tokeniza, busca términos de género y cuenta las palabras próximas. Añade además filtros que eliminan los términos objetivo, los *tokens* demasiado cortos, las *stopwords* estándar y el ruido típico de Twitter, como *rt*, *dm*, *http*, *https* o *tco*.

Esta función enseña una idea avanzada: el análisis real exige adaptar la limpieza al dominio. Las *stopwords* generales no siempre bastan, pero limpiar demasiado puede borrar señales discursivas, porque toda decisión técnica tiene un coste interpretativo.

2.21. Cómo escribir una interpretación responsable

La salida de este *pipeline* puede alimentar un informe, pero el informe debe formularse con precisión. Algunas formulaciones responsables serían:

- «El sistema ha detectado términos de género marcados en este subconjunto de textos».
- «El léxico valorativo aparece con mayor o menor frecuencia según las listas definidas».
- «Las coocurrencias sugieren asociaciones que conviene revisar con KWIC».
- «La etiqueta de sentimiento del conjunto de datos y la puntuación léxica no miden exactamente lo mismo».
- «No se infiere intención comunicativa a partir de estos resultados automáticos».

Algunas formulaciones que conviene evitar:

- «El algoritmo demuestra que el texto es sexista».
- «Si no aparece léxico valorativo, no hay valoración».
- «Una coocurrencia frecuente prueba una asociación discriminatoria».
- «La limpieza del texto es neutra».
- «La etiqueta del conjunto de datos es la verdad y nuestro análisis solo la confirma».

La competencia profesional reside precisamente en esta diferencia: programar es, además de obtener salidas, saber qué tipo de afirmación permite cada una.

Errores frecuentes de aprendizaje

- Confundir detección léxica con interpretación completa del sesgo.
- Pensar que una palabra de género es problemática por sí misma.
- Olvidar que las listas léxicas son decisiones humanas y deben revisarse.
- Interpretar una puntuación de polaridad como si midiera intención o ironía.
- Eliminar *stopwords* sin pensar qué información discursiva se pierde.
- Creer que un corpus filtrado representa todo el corpus original.
- Confundir la etiqueta externa de sentimiento con la valoración detectada por el prototipo.
- Leer una coocurrencia como prueba causal.
- No volver al texto original después de mirar tablas.
- Presentar resultados automáticos con lenguaje demasiado concluyente.

Síntesis final

Esta unidad presenta una forma responsable de usar Python para analizar el sesgo léxico y discursivo. Más que la promesa de una inteligencia artificial infalible, su punto de partida es una práctica lingüística transparente: definir qué se busca, preparar los textos, construir léxicos definidos de antemano, detectar marcas, contar patrones y volver al contexto.

El marco teórico muestra que el sesgo en PLN va más allá de un problema técnico, porque se relaciona con la representación social, los datos, la anotación, los modelos, los corpus y el poder. La investigación reciente insiste en que toda medición de sesgo requiere definiciones precisas, documentación de los datos y cautela interpretativa.

La parte de Python enseña el esqueleto de un *pipeline* reutilizable: DataFrame, normalización, tokenización, lematización opcional, *stopwords*, diccionarios, funciones de detección, tablas de resultados, frecuencias, KWIC, coocurrencias, descarga de datos reales, filtrado temático y agrupación. Cada operación produce una representación distinta del texto y, por tanto, una forma distinta de mirar el lenguaje.

El aprendizaje principal es que un análisis automático bien diseñado organiza la lectura humana en lugar de sustituirla: permite localizar indicios y formular preguntas, pero la interpretación lingüística sigue siendo el núcleo profesional del trabajo.

Cierre narrativo

Al terminar la simulación, Veridia Text Lab no entrega al cliente una lista de textos «culpables» o «inocentes», sino algo más serio: un prototipo explicable, un conjunto de criterios, una tabla de indicios y una guía de interpretación. El equipo puede mostrar qué palabras se han buscado, cómo se han limpiado los textos, qué frecuencia tiene cada término, qué coocurrencias aparecen y qué límites tiene el análisis.

Ese es el valor de una lingüista computacional en este tipo de proyecto: lejos de limitarse a ejecutar código o a repetir resultados, traduce entre tres mundos, el del lenguaje humano, el de la estructura computacional y el de la responsabilidad social de interpretar textos. Cuando el cliente pregunte si el sistema «detecta sesgo», la respuesta profesional será más precisa: detecta ciertos indicios léxicos y discursivos definidos de antemano, que deben revisarse con lectura experta.

En el aula, los casos prácticos permitirán aplicar este *pipeline* y discutir resultados concretos. Antes de llegar ahí, este manual te da la base conceptual y técnica para leer el código con criterio: saber qué hace cada bloque, qué presupone y qué tipo de conclusión permite.

Referencias

- Baker, P. (2006). *Using corpora in discourse analysis*. Continuum.
- Baker, P., Gabrielatos, C., KhosraviNik, M., Krzyżanowski, M., McEnery, T. y Wodak, R. (2008). A useful methodological synergy? Combining critical discourse analysis and corpus linguistics to examine discourses of refugees and asylum seekers in the UK press. *Discourse & Society*, 19(3), 273–306. <https://doi.org/10.1177/0957926508088962>
- Bender, E. M. y Friedman, B. (2018). Data statements for natural language processing: Toward mitigating system bias and enabling better science. *Transactions of the Association for Computational Linguistics*, 6, 587–604. https://doi.org/10.1162/tacl_a_00041
- Blodgett, S. L., Barocas, S., Daumé III, H. y Wallach, H. (2020). Language (technology) is power: A critical survey of "bias" in NLP. En *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 5454–5476). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.485>
- Bolukbasi, T., Chang, K.-W., Zou, J., Saligrama, V. y Kalai, A. (2016). Man is to computer programmer as woman is to homemaker? Debiasing word embeddings. En *Advances in Neural Information Processing Systems 29* (pp. 4349–4357). Curran Associates.
- Caliskan, A., Bryson, J. J. y Narayanan, A. (2017). Semantics derived automatically from language corpora contain human-like biases. *Science*, 356(6334), 183–186. <https://doi.org/10.1126/science.aal4230>
- Explosion AI. (2026). *Linguistic features*. spaCy Usage Documentation. <https://spacy.io/usage/linguistic-features>
- Gallegos, I. O., Rossi, R. A., Barrow, J., Tanjim, M. M., Kim, S., Deroncourt, F., Yu, T., Zhang, R. y Ahmed, N. K. (2024). Bias and fairness in large language models: A survey. *Computational Linguistics*, 50(3), 1097–1179. https://doi.org/10.1162/coli_a_00524
- Garrido-Muñoz, I., Martínez-Santiago, F. y Montejo-Ráez, A. (2024). MarIA and BETO are sexist: Evaluating gender bias in large language models for Spanish. *Language Resources and Evaluation*, 58(4), 1387–1417. <https://doi.org/10.1007/s10579-023-09670-3>
- Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H. y Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92. <https://doi.org/10.1145/3458723>
- Hu, M. y Liu, B. (2004). Mining and summarizing customer reviews. En *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 168–177). ACM. <https://doi.org/10.1145/1014052.1014073>
- Liu, B. (2012). *Sentiment analysis and opinion mining*. Morgan & Claypool. <https://doi.org/10.2200/S00416ED1V01Y201204HLT016>
- McEnery, T. y Hardie, A. (2012). *Corpus linguistics: Method, theory and practice*. Cambridge University Press.
- Mohammad, S. M. y Turney, P. D. (2013). Crowdsourcing a word-emotion association lexicon. *Computational Intelligence*, 29(3), 436–465. <https://doi.org/10.1111/j.1467-8640.2012.00460.x>
- Pérez, J. M., Furman, D. A., Alonso Alemany, L. y Luque, F. M. (2022). RoBERTuito: A pre-trained language model for social media text in Spanish. En *Proceedings of the Thirteenth Language Resources and Evaluation Conference* (pp. 7235–7243). European Language Resources Association. <https://aclanthology.org/2022.lrec-1.785/>
- Python Software Foundation. (2026). *re — Regular expression operations*. Python 3.14 Documentation. <https://docs.python.org/3/library/re.html>
- Sap, M., Card, D., Gabriel, S., Choi, Y. y Smith, N. A. (2019). The risk of racial bias in hate speech detection. En *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 1668–1678). Association for Computational Linguistics. <https://doi.org/10.18653/v1/P19-1163>
- Sheng, E., Chang, K.-W., Natarajan, P. y Peng, N. (2019). The woman worked as a babysitter: On biases in language generation. En *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the*

9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP) (pp. 3407–3412). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1339>